

Submission Date:
7 May 2026

Phase 3 – Physical Design



CMPG 311

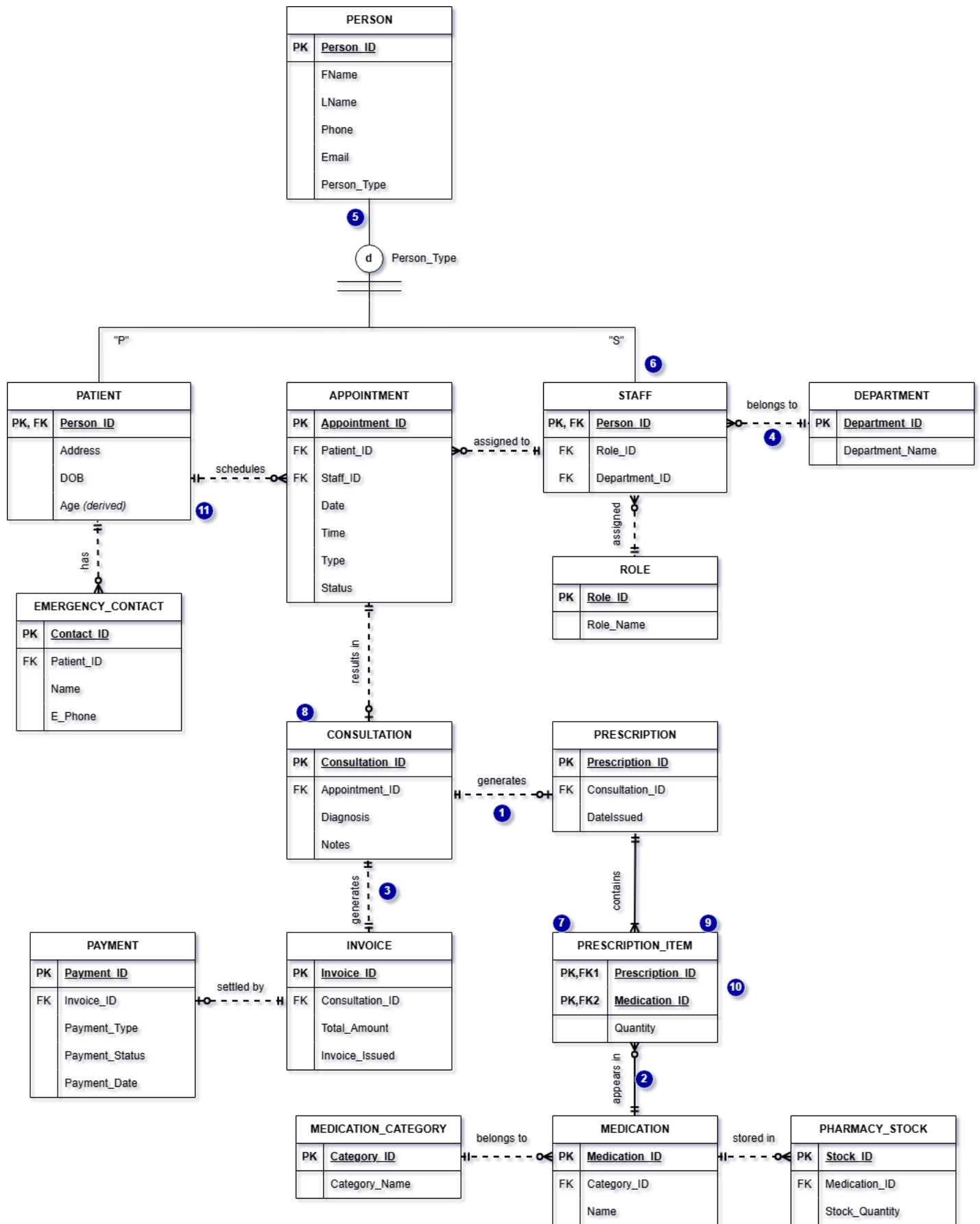
V.C. Freemantle	50333755
M. Tau	42027373
R. Zandberg	48907790
L. Zitha	38209098
B. Shaw	41311019

Table of Contents

Physical Data Model	3
Database Objects.....	4
Tables.....	4
Initialisation	4
Drop Tables	4
Drop Sequences	4
PERSON.....	5
Create, Constraints, Checks.....	5
ROLE.....	5
Create, Constraints, Checks.....	5
DEPARTMENT	5
Create, Constraints, Checks.....	5
MEDICATION_CATEGORY	6
Create, Constraints, Checks.....	6
PATIENT	6
Create, Constraints, Checks.....	6
STAFF.....	6
Create, Constraints, Checks.....	6
APPOINTMENT	7
Create, Constraints, Checks.....	7
CONSULTATION	7
Create, Constraints, Checks.....	7
PRESCRIPTION.....	8
Create, Constraints, Checks.....	8
MEDICATION	8
Create, Constraints, Checks.....	8
PRESCRIPTION_ITEM.....	8
Create, Constraints, Checks.....	8
PHARMACY_STOCK	9
Create, Constraints, Checks.....	9
INVOICE.....	9
Create, Constraints, Checks.....	9
PAYMENT	10
Create, Constraints, Checks.....	10
EMERGENCY_CONTACT	10
Create, Constraints, Checks.....	10

Sequences	11
Indexes.....	12
Triggers.....	13
Views	14
Data on Tables	15
Sequence Test Inserts	21
Queries	22
SELECT Statements	22
Based on information requirements of NexCare	22
Limitation of rows and columns	22
Sorting.....	23
LIKE, AND, OR.....	23
Variables and character functions	23
Round	23
Date functions	24
Aggregate functions	24
Group by and Having.....	24
Join.....	25
Sub-queries (Nested queries)	25
Extra Functionalities	26

Physical Data Model



Database Objects

Tables

Initialisation

Drop Tables

Remove any tables from the database before creating new versions of these tables.

```
1  -- =====
2  -- 1. DROP TABLES
3  -- =====
4
5  DROP TABLE PERSON CASCADE CONSTRAINTS;
6  DROP TABLE ROLE CASCADE CONSTRAINTS;
7  DROP TABLE DEPARTMENT CASCADE CONSTRAINTS;
8  DROP TABLE MEDICATION_CATEGORY CASCADE CONSTRAINTS;
9  DROP TABLE PATIENT CASCADE CONSTRAINTS;
10 DROP TABLE STAFF CASCADE CONSTRAINTS;
11 DROP TABLE APPOINTMENT CASCADE CONSTRAINTS;
12 DROP TABLE CONSULTATION CASCADE CONSTRAINTS;
13 DROP TABLE PRESCRIPTION CASCADE CONSTRAINTS;
14 DROP TABLE MEDICATION CASCADE CONSTRAINTS;
15 DROP TABLE PRESCRIPTION_ITEM CASCADE CONSTRAINTS;
16 DROP TABLE PHARMACY_STOCK CASCADE CONSTRAINTS;
17 DROP TABLE INVOICE CASCADE CONSTRAINTS;
18 DROP TABLE PAYMENT CASCADE CONSTRAINTS;
19 DROP TABLE EMERGENCY_CONTACT CASCADE CONSTRAINTS;
```

Drop Sequences

Remove any sequences from the database before creating new versions of these sequences.

```
22 -- =====
23 -- 2. DROP SEQUENCES
24 -- =====
25 DROP SEQUENCE seq_person;
26 DROP SEQUENCE seq_appointment;
27 DROP SEQUENCE seq_consultation;
28 DROP SEQUENCE seq_prescription;
29 DROP SEQUENCE seq_invoice;
30 DROP SEQUENCE seq_payment;
```

PERSON

The Person table is a supertype of Patient and Staff. It contains shared attributes between patients and staff, such as Name, Surname, Phone number, and email address. The relationship from Person to Patient and Staff is disjoint, meaning that a person can only be one of the two.

Create, Constraints, Checks

```
37  /*Person*/
38  CREATE TABLE PERSON(
39      Person_ID NUMBER PRIMARY KEY,
40      FName VARCHAR2(50) NOT NULL,
41      LName VARCHAR2(50) NOT NULL,
42      Phone VARCHAR2(10) NOT NULL,
43      Email VARCHAR2(50) NOT NULL,
44      Person_Type VARCHAR2(1) NOT NULL
45
46      CONSTRAINT chk_person_type CHECK (Person_Type IN ('P', 'S')),
47
48      CONSTRAINT chk_phoneNum_length
49          CHECK (LENGTH(Phone) >= 10)
50  );
```

ROLE

The Role table stores information about the different job roles within the healthcare system, such as accountant, nurse, or pharmacist. Each role has a unique Role_ID (primary key) and a Role_Name that describes the type of staff position. This table is used to define and standardise staff responsibilities within the organisation.

Create, Constraints, Checks

```
53  /*Role*/
54  CREATE TABLE ROLE(
55      Role_ID NUMBER PRIMARY KEY,
56      Role_Name VARCHAR2(50) NOT NULL
57  );
```

DEPARTMENT

The Department table stores information about the different departments within the healthcare facility, such as emergency, pharmacy, or outpatient services. Each department is uniquely identified by a Department_ID (primary key) and includes a Department_Name. This table helps organise staff and operations according to functional areas within the hospital.

Create, Constraints, Checks

```
60  /*Department*/
61  CREATE TABLE DEPARTMENT(
62      Department_ID NUMBER PRIMARY KEY,
63      Department_Name VARCHAR2(50) NOT NULL
64  );
```

MEDICATION_CATEGORY

The Medication_Category table stores classification information for medications used in the system. Each category has a unique Category_ID (primary key) and a Category_Name that describes the type of medication group, such as antibiotics or painkillers. This table ensures medications are organised into logical groups for easier management and reporting.

Create, Constraints, Checks

```
67  /*Medication Category*/
68  CREATE TABLE MEDICATION_CATEGORY (
69      Category_ID NUMBER PRIMARY KEY,
70      Category_Name VARCHAR2(25) NOT NULL
71  );
```

PATIENT

The Patient table stores patient-specific information such as date of birth and address. Each patient is uniquely identified by Patient_ID, which also links to the Person table through a foreign key relationship. This table extends the Person entity and contains medical-related demographic information required for healthcare management.

Create, Constraints, Checks

```
74  /*Patient*/
75  CREATE TABLE PATIENT(
76      Person_ID NUMBER PRIMARY KEY,
77      Address VARCHAR2(100) NOT NULL,
78      DOB DATE NOT NULL,
79
80      CONSTRAINT fk_patient
81          FOREIGN KEY(Person_ID) REFERENCES PERSON(Person_ID)
82  );
```

STAFF

The Staff table stores information about employees working within the healthcare facility. Each staff member is uniquely identified by Person_ID (primary key) and is linked to the Person table through a foreign key relationship. It also includes Role_ID and Department_ID foreign keys to define the staff member's role and department. This table is essential for managing employee responsibilities and linking staff to appointments and consultations.

Create, Constraints, Checks

```
84  /*Staff*/
85  CREATE TABLE STAFF(
86      Person_ID NUMBER PRIMARY KEY,
87      Role_ID NUMBER NOT NULL,
88      Department_ID NUMBER NOT NULL,
89
90      CONSTRAINT fk_person
91          FOREIGN KEY(Person_ID) REFERENCES PERSON(Person_ID),
92
93      CONSTRAINT fk_role_id
94          FOREIGN KEY(Role_ID) REFERENCES ROLE(Role_ID),
95
96      CONSTRAINT fk_department_id
97          FOREIGN KEY(Department_ID) REFERENCES DEPARTMENT(Department_ID)
98  );
```

APPOINTMENT

The Appointment table stores information about scheduled meetings between patients and staff. Each appointment has a unique Appointment_ID (primary key) and includes details such as date, time, type, and status. It contains two foreign keys: Patient_ID and Staff_ID, which link the appointment to a specific patient and staff member. All important fields are mandatory, ensuring complete and valid records.

Create, Constraints, Checks

```
100  /*Appointment*/
101  CREATE TABLE APPOINTMENT (
102      Appointment_ID NUMBER PRIMARY KEY,
103      Patient_ID NUMBER NOT NULL,
104      Staff_ID NUMBER NOT NULL,
105      App_Date DATE NOT NULL,
106      App_Time VARCHAR2(10) NOT NULL,
107      App_Type VARCHAR2(50) NOT NULL,
108      Status VARCHAR2(20) NOT NULL,
109
110      CONSTRAINT fk_appointment_patient
111          FOREIGN KEY (Patient_ID) REFERENCES PATIENT(Person_ID),
112
113      CONSTRAINT fk_appointment_staff
114          FOREIGN KEY (Staff_ID) REFERENCES STAFF(Person_ID),
115
116      CONSTRAINT chk_status
117          CHECK (Status IN ('Scheduled', 'Completed', 'Cancelled', 'No Show')),
118
119      CONSTRAINT chk_app_type
120          CHECK (App_Type IN ('Consultation', 'Follow-up', 'Emergency', 'Prescription Refill'))
121  );
```

CONSULTATION

The Consultation table stores information about medical consultations that occur after an appointment. Each consultation is uniquely identified by the Consultation_ID (primary key). It includes details such as the diagnosis and any additional notes recorded during the consultation. The table contains a foreign key, Appointment_ID, which links each consultation to a specific appointment.

Create, Constraints, Checks

```
123  /*Consultation*/
124  CREATE TABLE CONSULTATION (
125      Consultation_ID NUMBER PRIMARY KEY,
126      Appointment_ID NUMBER NOT NULL,
127      Diagnosis VARCHAR2(200),
128      Notes VARCHAR2(300),
129
130      CONSTRAINT fk_consultation_appointment
131          FOREIGN KEY (Appointment_ID) REFERENCES APPOINTMENT(Appointment_ID)
132  );
```

PRESCRIPTION

The Prescription table stores information about medications prescribed to patients during a consultation. Each prescription has a unique Prescription_ID (primary key) and is linked to a specific consultation through the Consultation_ID foreign key. It also includes the date the prescription was issued. This table ensures that all prescribed treatments are properly recorded and traceable back to a medical consultation.

Create, Constraints, Checks

```
136 /*Prescription*/
137 CREATE TABLE PRESCRIPTION(
138     Prescription_ID NUMBER PRIMARY KEY,
139     Consultation_ID NUMBER NOT NULL,
140     DateIssued DATE NOT NULL,
141
142     CONSTRAINT consultationID_fk
143     FOREIGN KEY (Consultation_ID) REFERENCES CONSULTATION (Consultation_ID)
144 );
```

MEDICATION

The Medication table stores details about all available medicines in the system. Each medication has a unique Medication_ID (primary key) and includes the medication name and category reference. This table acts as the central reference for all prescribed drugs and pharmacy stock management.

Create, Constraints, Checks

```
147 /*Medication*/
148 CREATE TABLE MEDICATION (
149     Medication_ID NUMBER PRIMARY KEY,
150     Category_ID NUMBER NOT NULL,
151     Name VARCHAR2(35) NOT NULL,
152
153     CONSTRAINT categoryID_fk
154     FOREIGN KEY (Category_ID) REFERENCES MEDICATION_CATEGORY(Category_ID)
155 );
```

PRESCRIPTION_ITEM

The Prescription_Item table represents the relationship between prescriptions and medications. It allows a single prescription to contain multiple medications, and each medication can appear in multiple prescriptions. The table includes a composite primary key consisting of Prescription_ID and Medication_ID. It also records the quantity of each medication prescribed. This table ensures accurate tracking of prescribed medication details.

Create, Constraints, Checks

```
158 /*Prescription item*/
159 CREATE TABLE PRESCRIPTION_ITEM(
160     Prescription_ID NUMBER,
161     Medication_ID NUMBER,
162     Quantity NUMBER NOT NULL,
163
164     CONSTRAINT prescriptionItem_pk
165     PRIMARY KEY (Prescription_ID, Medication_ID),
166
167     CONSTRAINT prescriptionID_fk FOREIGN KEY (Prescription_ID)
168     REFERENCES PRESCRIPTION(Prescription_ID),
169
170     CONSTRAINT medicationID_fk FOREIGN KEY (Medication_ID)
171     REFERENCES MEDICATION(Medication_ID)
172 );
```

PHARMACY_STOCK

The Pharmacy_Stock table stores information about the availability of medications in the pharmacy. Each record is uniquely identified by Stock_ID and is linked to a Medication_ID through a foreign key. It includes the current stock quantity of each medication. This table helps the system track inventory levels and manage medication availability.

Create, Constraints, Checks

```
176  /*Pharmacy Stock*/
177  CREATE TABLE PHARMACY_STOCK(
178      Stock_ID NUMBER PRIMARY KEY,
179      Medication_ID NUMBER NOT NULL,
180      Stock_Quantity NUMBER NOT NULL,
181
182      CONSTRAINT fk_stock_medicationID
183          FOREIGN KEY (Medication_ID) REFERENCES MEDICATION(Medication_ID),
184
185      CONSTRAINT stock_quantity_min CHECK (Stock_Quantity >= 0)
186  );
```

INVOICE

The Invoice table stores billing information generated after a consultation. Each invoice has a unique Invoice_ID (primary key) and is linked to a specific consultation through the Consultation_ID foreign key. It includes the total amount charged and the date the invoice was issued. This table is used to manage patient billing and financial records.

Create, Constraints, Checks

```
188  /*Invoice*/
189  CREATE TABLE INVOICE (
190      Invoice_ID NUMBER PRIMARY KEY,
191      Consultation_ID NUMBER NOT NULL,
192      Total_Amount NUMBER(10, 2) NOT NULL,
193      Invoice_Issued DATE NOT NULL,
194
195      CONSTRAINT fk_invoice_consultation
196          FOREIGN KEY (Consultation_ID) REFERENCES CONSULTATION(Consultation_ID),
197
198      CHECK (Total_Amount > 0)
199  );
```

PAYMENT

The Payment table stores information about payments made against invoices. Each payment has a unique Payment_ID and is linked to an Invoice_ID through a foreign key. It includes details such as payment type (cash, card, EFT, and Medical aid), payment status (paid or pending), and payment date. This table ensures accurate tracking of all financial transactions.

Create, Constraints, Checks

```
201  /*Payment*/
202  CREATE TABLE PAYMENT(
203      Payment_ID NUMBER PRIMARY KEY,
204      Invoice_ID NUMBER NOT NULL,
205      Payment_Type VARCHAR2(20) NOT NULL,
206      Payment_Status VARCHAR2(20) NOT NULL,
207      Payment_Date DATE,
208
209      CONSTRAINT fk_payment_invoice
210          FOREIGN KEY (Invoice_ID) REFERENCES INVOICE(Invoice_ID),
211
212      CONSTRAINT Payment_Invoice_Unique UNIQUE (Invoice_ID),
213
214      CONSTRAINT chk_payment_status
215          CHECK (Payment_Status IN ('Paid', 'Pending')),
216
217      CONSTRAINT chk_payment_type
218          CHECK (Payment_Type IN ('Card', 'Cash', 'EFT', 'Medical Aid', 'N/A'))
219  );
```

EMERGENCY_CONTACT

The EMERGENCY_CONTACT table stores essential contact information for individuals who should be notified in case of a medical emergency involving a patient. Each entry is linked to a specific patient and includes the contact's name and phone number. This table ensures that the healthcare facility has quick access to reliable emergency communication details.

Create, Constraints, Checks

```
221  /*Emergency Contact*/
222  CREATE TABLE EMERGENCY_CONTACT(
223      Contact_ID NUMBER PRIMARY KEY,
224      Patient_ID NUMBER NOT NULL,
225      Name VARCHAR2(50) NOT NULL,
226      E_Phone VARCHAR2(10) NOT NULL,
227
228      CONSTRAINT fk_contact_patient
229          FOREIGN KEY (Patient_ID) REFERENCES PATIENT(Person_ID),
230
231      CONSTRAINT chk_phone_length
232          CHECK (LENGTH(E_Phone) >= 10)
233  );
```

Sequences

This section shows the creation of sequences used to automatically generate unique primary key values for key tables such as Person, Appointment, Consultation, Prescription, Invoice, and Payment. These sequences ensure data integrity and eliminate the need for manual ID entry during data insertion.

```
236 -- =====
237 -- 4. SEQUENCES
238 -- =====
239 CREATE SEQUENCE seq_person
240     START WITH 508
241     INCREMENT BY 1;
242
243 CREATE SEQUENCE seq_appointment
244     START WITH 21
245     INCREMENT BY 1;
246
247 CREATE SEQUENCE seq_consultation
248     START WITH 13
249     INCREMENT BY 1;
250
251 CREATE SEQUENCE seq_prescription
252     START WITH 213
253     INCREMENT BY 1;
254
255 CREATE SEQUENCE seq_invoice
256     START WITH 1013
257     INCREMENT BY 1;
258
259 CREATE SEQUENCE seq_payment
260     START WITH 2013
261     INCREMENT BY 1;
```

Indexes

This section shows the creation of indexes on frequently searched columns such as foreign keys and date fields. These indexes improve query performance by allowing faster data retrieval when joining tables or filtering large datasets.

```
601 -- =====
602 -- 8. INDEXES
603 -- =====
604
605 /*Staff*/
606 CREATE INDEX staffRole_idx ON STAFF(Role_ID);
607 CREATE INDEX staffDepartment_idx ON STAFF(Department_ID);
608
609 /*Appointment*/
610 CREATE INDEX appointmentPatient_idx ON APPOINTMENT(Patient_ID);
611 CREATE INDEX appointmentStaff_idx ON APPOINTMENT(Staff_ID);
612 CREATE INDEX appointmentDate_idx ON APPOINTMENT(App_Date);
613
614 /*Consultation*/
615 CREATE INDEX consultationAppointment_idx ON CONSULTATION(Appointment_ID);
616
617 /*Invoice*/
618 CREATE INDEX invoiceConsultation_idx ON INVOICE(Consultation_ID);
619
620 /*Prescription*/
621 CREATE INDEX prescriptionConsultation_idx ON PRESCRIPTION(Consultation_ID);
622
623 /*Prescription_Item*/
624 CREATE INDEX prescriptionItemMedication_idx ON PRESCRIPTION_ITEM(Medication_ID);
625
626 /*Medication*/
627 CREATE INDEX medicationCategory_idx ON MEDICATION(Category_ID);
628
629 /*Pharmacy_Stock*/
630 CREATE INDEX stockMedication_idx ON PHARMACY_STOCK(Medication_ID);
631
632 /*Emergency_Contact*/
633 CREATE INDEX contactPatient_idx ON EMERGENCY_CONTACT(Patient_ID);
```

Triggers

This section enforces automatic rules in the database to ensure data consistency and validate important business logic.

Automatically sets payment status to Paid or Pending based on whether a payment date is entered.

```
697 -- =====
698 -- 10. TRIGGERS
699 -- =====
700
701 /*Payment status set automatically based on payment type*/
702 CREATE OR REPLACE TRIGGER trg_payment_status
703 BEFORE INSERT ON PAYMENT
704 FOR EACH ROW
705 BEGIN
706     IF :NEW.Payment_Date IS NULL THEN
707         :NEW.Payment_Status := 'Pending';
708     ELSE
709         :NEW.Payment_Status := 'Paid';
710     END IF;
711 END;
712 /
```

Prevents invalid stock values by ensuring that medication stock cannot be negative when inserting or updating records.

```
714 /*Ensures stock is a positive number*/
715 CREATE OR REPLACE TRIGGER trg_stock_check
716 BEFORE INSERT OR UPDATE ON PHARMACY_STOCK
717 FOR EACH ROW
718 BEGIN
719     IF :NEW.Stock_Quantity < 0 THEN
720         RAISE_APPLICATION_ERROR(-20001, 'Stock cannot be negative');
721     END IF;
722 END;
723 /
```

Views

This section creates simplified “saved queries” that combine multiple tables so that complex NexCare information can be viewed easily without rewriting full joins each time.

Displays all patient appointments together with personal details and appointment information in one combined view.

```
638 /*All Patient Appointments*/
639 CREATE OR REPLACE VIEW get_all_patient_appointments AS
640 SELECT
641     p.Person_ID, per.FName, per.LName, per.Phone, a.Appointment_ID, a.App_Date, a.App_Time, a.App_Type, a.Status
642 FROM PATIENT p
643 JOIN PERSON per ON p.Person_ID = per.Person_ID
644 JOIN APPOINTMENT a ON p.Person_ID = a.Patient_ID;
```

Shows only appointments that have been completed, linking patients with their consultation records.

```
646 /*All Completed Patient Appointments*/
647 CREATE OR REPLACE VIEW get_all_patient_appointments AS
648 SELECT
649     p.Person_ID, per.FName, per.LName, per.Phone, a.Appointment_ID, a.App_Date, a.App_Time, a.App_Type, a.Status
650 FROM PATIENT p
651 JOIN PERSON per ON p.Person_ID = per.Person_ID
652 JOIN APPOINTMENT a ON p.Person_ID = a.Patient_ID
653 WHERE a.Status = 'Completed';
```

Combines consultation and appointment data to show all consultation activity and related appointment details.

```
655 /*Active Consultations*/
656 CREATE OR REPLACE VIEW get_all_consultations AS
657 SELECT
658     c.Consultation_ID, c.Appointment_ID, c.Diagnosis, c.Notes, a.App_Date, a.App_Type, a.Status
659 FROM CONSULTATION c
660 JOIN APPOINTMENT a ON c.Appointment_ID = a.Appointment_ID;
```

Displays prescription details together with diagnosis information for easier tracking of treatment given to patients.

```
662 /*Prescription summary*/
663 CREATE OR REPLACE VIEW get_all_prescriptions AS
664 SELECT
665     pr.Prescription_ID, pr.Consultation_ID, c.Diagnosis, pr.DateIssued
666 FROM PRESCRIPTION pr
667 JOIN CONSULTATION c ON pr.Consultation_ID = c.Consultation_ID;
```

Shows medication stock levels and classifies items as in stock, low stock, or out of stock for inventory monitoring.

```
669 /*Pharmacy stock*/
670 CREATE OR REPLACE VIEW get_all_pharmacy_stock AS
671 SELECT
672     m.Medication_ID, m.Name, ps.Stock_Quantity,
673     CASE
674         WHEN ps.Stock_Quantity = 0 THEN 'Out of Stock'
675         WHEN ps.Stock_Quantity < 50 THEN 'Low Stock'
676         ELSE 'In Stock'
677     END AS Stock_Status
678 FROM MEDICATION m
679 JOIN PHARMACY_STOCK ps ON m.Medication_ID = ps.Medication_ID;
```

Combines patient, invoice, and payment information to give a full overview of billing and payment history per patient.

```
681 /*Patient Billing Summary*/
682 CREATE OR REPLACE VIEW get_patient_billing_summary AS
683 SELECT
684     p.Person_ID, per.FName, per.LName, i.Invoice_ID, i.Total_Amount, i.Invoice_Issued, pay.Payment_Type,
685     pay.Payment_Status, pay.Payment_Date
686 FROM PATIENT p
687 JOIN PERSON per
688     ON p.Person_ID = per.Person_ID
689 JOIN CONSULTATION c
690     ON p.Person_ID = c.Appointment_ID
691 JOIN INVOICE i
692     ON c.Consultation_ID = i.Consultation_ID
693 LEFT JOIN PAYMENT pay
694     ON i.Invoice_ID = pay.Invoice_ID;
```

Data on Tables

This section presents database views that simplify complex queries by combining multiple tables into a single virtual table. The views provide structured outputs such as patient appointments, consultations, prescriptions, pharmacy stock status, and billing summaries for easier reporting and analysis.

Person

```
264 -- =====
265 -- 5. INSERT DATA
266 -- =====
267 /*Insert data into PERSON(Staff) table*/
268 INSERT INTO PERSON VALUES (101, 'Thabo', 'Molefe', '0721234567', 'thabo@NexCare.co.za', 'S');
269 INSERT INTO PERSON VALUES (102, 'Sarah', 'Smith', '0839876543', 'sarah.s@NexCare.co.za', 'S');
270 INSERT INTO PERSON VALUES (103, 'Pieter', 'Botha', '0112345678', 'pieter.b@NexCare.co.za', 'S');
271 INSERT INTO PERSON VALUES (104, 'Nomsa', 'Dlamini', '0615551234', 'nomsa.d@NexCare.co.za', 'S');
272 INSERT INTO PERSON VALUES (105, 'Elena', 'Naidoo', '0748889999', 'elena.n@NexCare.co.za', 'S');
273 INSERT INTO PERSON VALUES (106, 'Michael', 'Adams', '0823456789', 'michael.a@nexcare.co.za', 'S');
274 INSERT INTO PERSON VALUES (107, 'Lindiwe', 'Maseko', '0719988776', 'lindiwe.m@nexcare.co.za', 'S');
275
276 /*Insert data into PERSON(Patient) table*/
277 INSERT INTO PERSON VALUES (501, 'John', 'Doe', '0821112222', 'johndoe@gmail.com', 'P');
278 INSERT INTO PERSON VALUES (502, 'Jane', 'Smit', '0733334444', 'jane.s@yahoo.com', 'P');
279 INSERT INTO PERSON VALUES (503, 'Zanele', 'Khumalo', '0845556666', 'zanele@mweb.co.za', 'P');
280 INSERT INTO PERSON VALUES (504, 'Kevin', 'Louw', '0657778888', 'kevin.l@telkom.sa', 'P');
281 INSERT INTO PERSON VALUES (505, 'Mary', 'Peters', '0129990000', 'mpeters@gmail.com', 'P');
282 INSERT INTO PERSON VALUES (506, 'Brian', 'Hendricks', '0831122334', 'brian.h@gmail.com', 'P');
283 INSERT INTO PERSON VALUES (507, 'Ayesha', 'Patel', '0722233445', 'ayasha.p@gmail.com', 'P');
```

Role

```
286 /*Insert data into ROLE table*/
287 INSERT INTO ROLE VALUES(1,'Physician');
288 INSERT INTO ROLE VALUES(2,'Nurse');
289 INSERT INTO ROLE VALUES(3,'Accountant');
290 INSERT INTO ROLE VALUES(4,'Pharmacist');
291 INSERT INTO ROLE VALUES(5,'Receptionist');
```

Department

```
294 /*Insert data into DEPARTMENT table*/
295 INSERT INTO DEPARTMENT VALUES(1,'Pharmacy');
296 INSERT INTO DEPARTMENT VALUES(2,'Administration');
297 INSERT INTO DEPARTMENT VALUES(3,'Consultation');
298 INSERT INTO DEPARTMENT VALUES(4,'Finance');
299 INSERT INTO DEPARTMENT VALUES(5,'Reception');
```

Medication_Category

```
302 /*Insert data into MEDICATION_CATEGORY table*/
303 INSERT INTO MEDICATION_CATEGORY VALUES (1, 'Antibiotics');
304 INSERT INTO MEDICATION_CATEGORY VALUES (2, 'Cardiovascular');
305 INSERT INTO MEDICATION_CATEGORY VALUES (3, 'Central Nervous System');
306 INSERT INTO MEDICATION_CATEGORY VALUES (4, 'Narcotic');
307 INSERT INTO MEDICATION_CATEGORY VALUES (5, 'Gastrointestinal');
308 INSERT INTO MEDICATION_CATEGORY VALUES (6, 'Respiratory');
```

Patient

```
311  /*Insert data into PATIENT table*/
312  INSERT INTO PATIENT VALUES (501, '12 Flower Rd, JHB', TO_DATE('1990-05-10', 'YYYY-MM-DD'));
313  INSERT INTO PATIENT VALUES (502, '45 Park Str, CPT', TO_DATE('1985-11-22', 'YYYY-MM-DD'));
314  INSERT INTO PATIENT VALUES (503, '7 Nelson Mandela Ave, JHB', TO_DATE('1998-01-30', 'YYYY-MM-DD'));
315  INSERT INTO PATIENT VALUES (504, '22 Pendoring Rd, DBN', TO_DATE('2002-07-15', 'YYYY-MM-DD'));
316  INSERT INTO PATIENT VALUES (505, '88 West Rd, JHB', TO_DATE('1975-03-05', 'YYYY-MM-DD'));
317  INSERT INTO PATIENT VALUES (506, '15 Sandton Dr, JHB', TO_DATE('1992-08-14', 'YYYY-MM-DD'));
318  INSERT INTO PATIENT VALUES (507, '9 Cape Rd, CPT', TO_DATE('1980-03-21', 'YYYY-MM-DD'));
```

Staff

```
321  /*Insert data into STAFF table*/
322  INSERT INTO STAFF VALUES (101, 1, 3);
323  INSERT INTO STAFF VALUES (102, 2, 3);
324  INSERT INTO STAFF VALUES (103, 3, 4);
325  INSERT INTO STAFF VALUES (104, 4, 1);
326  INSERT INTO STAFF VALUES (105, 5, 5);
327  INSERT INTO STAFF VALUES (106, 1, 3);
328  INSERT INTO STAFF VALUES (107, 2, 5);
```

Appointment

```
331  /*Insert data into APPOINTMENT table*/
332  INSERT INTO APPOINTMENT VALUES (1, 501, 101, TO_DATE('2026-02-02', 'YYYY-MM-DD'), '09:00', 'Consultation', 'Completed');
333  INSERT INTO APPOINTMENT VALUES (2, 502, 102, TO_DATE('2026-05-08', 'YYYY-MM-DD'), '09:30', 'Follow-up', 'Cancelled');
334  INSERT INTO APPOINTMENT VALUES (3, 503, 103, TO_DATE('2026-04-20', 'YYYY-MM-DD'), '10:00', 'Emergency', 'Completed');
335  INSERT INTO APPOINTMENT VALUES (4, 504, 104, TO_DATE('2026-06-12', 'YYYY-MM-DD'), '10:30', 'Prescription Refill', 'Scheduled');
336
337  INSERT INTO APPOINTMENT VALUES (5, 505, 105, TO_DATE('2026-04-02', 'YYYY-MM-DD'), '09:00', 'Consultation', 'Completed');
338  INSERT INTO APPOINTMENT VALUES (6, 506, 101, TO_DATE('2026-05-04', 'YYYY-MM-DD'), '09:30', 'Follow-up', 'No Show');
339  INSERT INTO APPOINTMENT VALUES (7, 507, 102, TO_DATE('2026-03-13', 'YYYY-MM-DD'), '10:00', 'Emergency', 'Completed');
340  INSERT INTO APPOINTMENT VALUES (8, 503, 103, TO_DATE('2026-01-28', 'YYYY-MM-DD'), '10:30', 'Prescription Refill', 'Completed');
341
342  INSERT INTO APPOINTMENT VALUES (9, 504, 104, TO_DATE('2026-06-25', 'YYYY-MM-DD'), '09:00', 'Consultation', 'Scheduled');
343  INSERT INTO APPOINTMENT VALUES (10, 501, 105, TO_DATE('2026-05-11', 'YYYY-MM-DD'), '09:30', 'Emergency', 'Cancelled');
344  INSERT INTO APPOINTMENT VALUES (11, 502, 101, TO_DATE('2026-05-01', 'YYYY-MM-DD'), '10:00', 'Follow-up', 'Completed');
345  INSERT INTO APPOINTMENT VALUES (12, 503, 102, TO_DATE('2026-03-19', 'YYYY-MM-DD'), '10:30', 'Consultation', 'Completed');
346
347  INSERT INTO APPOINTMENT VALUES (13, 504, 103, TO_DATE('2026-02-10', 'YYYY-MM-DD'), '09:00', 'Prescription Refill', 'Completed');
348  INSERT INTO APPOINTMENT VALUES (14, 505, 104, TO_DATE('2026-04-30', 'YYYY-MM-DD'), '09:30', 'Consultation', 'Cancelled');
349  INSERT INTO APPOINTMENT VALUES (15, 506, 105, TO_DATE('2026-04-15', 'YYYY-MM-DD'), '10:00', 'Follow-up', 'Completed');
350  INSERT INTO APPOINTMENT VALUES (16, 507, 101, TO_DATE('2026-03-05', 'YYYY-MM-DD'), '10:30', 'Emergency', 'Completed');
351
352  INSERT INTO APPOINTMENT VALUES (17, 505, 102, TO_DATE('2026-05-21', 'YYYY-MM-DD'), '09:00', 'Consultation', 'No Show');
353  INSERT INTO APPOINTMENT VALUES (18, 507, 103, TO_DATE('2026-05-05', 'YYYY-MM-DD'), '09:30', 'Follow-up', 'Completed');
354  INSERT INTO APPOINTMENT VALUES (19, 501, 104, TO_DATE('2026-01-15', 'YYYY-MM-DD'), '10:00', 'Emergency', 'Completed');
355  INSERT INTO APPOINTMENT VALUES (20, 502, 105, TO_DATE('2026-06-02', 'YYYY-MM-DD'), '10:30', 'Prescription Refill', 'Scheduled');
```

Consultation

```
358 /*Insert data into CONSULTATION table*/
359 INSERT INTO CONSULTATION VALUES (1, 1, 'Hypertension', 'BP monitoring advised.');
```

360	INSERT INTO CONSULTATION VALUES (2, 3, 'Asthma Review', 'Inhaler adjusted.');
361	INSERT INTO CONSULTATION VALUES (3, 5, 'Flu Symptoms', 'Rest and fluids recommended.');
362	INSERT INTO CONSULTATION VALUES (4, 7, 'Emergency Trauma', 'Immediate treatment provided.');
363	INSERT INTO CONSULTATION VALUES (5, 8, 'Prescription Review', 'Medication updated.');
364	INSERT INTO CONSULTATION VALUES (6, 11, 'Follow-up Check', 'Patient improving.');
365	
366	INSERT INTO CONSULTATION VALUES (7, 12, 'Routine Consultation', 'General health stable.');
367	INSERT INTO CONSULTATION VALUES (8, 13, 'Chronic Care Review', 'Ongoing management.');
368	INSERT INTO CONSULTATION VALUES (9, 15, 'Back Pain', 'Physiotherapy recommended.');
369	INSERT INTO CONSULTATION VALUES (10, 16, 'Emergency Case', 'Stabilised in ER.');
370	INSERT INTO CONSULTATION VALUES (11, 18, 'Follow-up Visit', 'Recovery on track.');
371	INSERT INTO CONSULTATION VALUES (12, 19, 'Critical Care Review', 'Condition monitored.');

Prescription

```
374 /*Insert data into PRESCRIPTION table*/
375 INSERT INTO PRESCRIPTION VALUES (201, 1, TO_DATE('2026-02-02','YYYY-MM-DD'));
376 INSERT INTO PRESCRIPTION VALUES (202, 2, TO_DATE('2026-04-20','YYYY-MM-DD'));
377 INSERT INTO PRESCRIPTION VALUES (203, 3, TO_DATE('2026-04-02','YYYY-MM-DD'));
378 INSERT INTO PRESCRIPTION VALUES (204, 4, TO_DATE('2026-03-13','YYYY-MM-DD'));
379 INSERT INTO PRESCRIPTION VALUES (205, 5, TO_DATE('2026-01-28','YYYY-MM-DD'));
380 INSERT INTO PRESCRIPTION VALUES (206, 6, TO_DATE('2026-05-01','YYYY-MM-DD'));
381
382 INSERT INTO PRESCRIPTION VALUES (207, 7, TO_DATE('2026-03-19','YYYY-MM-DD'));
383 INSERT INTO PRESCRIPTION VALUES (208, 8, TO_DATE('2026-02-10','YYYY-MM-DD'));
384 INSERT INTO PRESCRIPTION VALUES (209, 9, TO_DATE('2026-04-15','YYYY-MM-DD'));
385 INSERT INTO PRESCRIPTION VALUES (210, 10, TO_DATE('2026-03-05','YYYY-MM-DD'));
386 INSERT INTO PRESCRIPTION VALUES (211, 11, TO_DATE('2026-05-05','YYYY-MM-DD'));
387 INSERT INTO PRESCRIPTION VALUES (212, 12, TO_DATE('2026-01-15','YYYY-MM-DD'));
```

Medication

```
390 /*Insert data into MEDICATION table*/
391 INSERT INTO MEDICATION VALUES (101, 1, 'Amoxicillin');
392 INSERT INTO MEDICATION VALUES (102, 1, 'Ciprofloxacin');
393 INSERT INTO MEDICATION VALUES (103, 1, 'Penicillin');
394
395 INSERT INTO MEDICATION VALUES (104, 2, 'Eliquis');
396 INSERT INTO MEDICATION VALUES (105, 2, 'Atenolol');
397 INSERT INTO MEDICATION VALUES (106, 2, 'Lisinopril');
398
399 INSERT INTO MEDICATION VALUES (107, 3, 'Gabapentin');
400 INSERT INTO MEDICATION VALUES (108, 3, 'Sertraline');
401 INSERT INTO MEDICATION VALUES (109, 3, 'Diazepam');
402
403 INSERT INTO MEDICATION VALUES (110, 4, 'Acetaminophen');
404 INSERT INTO MEDICATION VALUES (111, 4, 'Morphine');
405 INSERT INTO MEDICATION VALUES (112, 4, 'Codeine');
406
407 INSERT INTO MEDICATION VALUES (113, 5, 'Prilosec');
408 INSERT INTO MEDICATION VALUES (114, 5, 'Loperamide');
409 INSERT INTO MEDICATION VALUES (115, 5, 'Metoclopramide');
410
411 INSERT INTO MEDICATION VALUES (116, 6, 'Albuterol');
412 INSERT INTO MEDICATION VALUES (117, 6, 'Budesonide');
413 INSERT INTO MEDICATION VALUES (118, 6, 'Ipratropium');
```

Prescription_Item

```
416 /*Insert data into PRESCRIPTION_ITEM table*/
417 /*Hypertension (Cardiovascular)*/
418 INSERT INTO PRESCRIPTION_ITEM VALUES (201, 104, 1);
419 INSERT INTO PRESCRIPTION_ITEM VALUES (201, 105, 1);
420
421 /*Asthma Review (Respiratory)*/
422 INSERT INTO PRESCRIPTION_ITEM VALUES (202, 116, 1);
423 INSERT INTO PRESCRIPTION_ITEM VALUES (202, 117, 1);
424
425 /*Flu Symptoms (Supportive / mild antibiotics optional)*/
426 INSERT INTO PRESCRIPTION_ITEM VALUES (203, 101, 1);
427 INSERT INTO PRESCRIPTION_ITEM VALUES (203, 114, 1);
428
429 /*Emergency Trauma (Pain + Antibiotics)*/
430 INSERT INTO PRESCRIPTION_ITEM VALUES (204, 111, 1);
431 INSERT INTO PRESCRIPTION_ITEM VALUES (204, 102, 1);
432
433 /*Prescription Review (CNS / pain management adjustment)*/
434 INSERT INTO PRESCRIPTION_ITEM VALUES (205, 107, 1);
435
436 /*Follow-up Check (Cardio maintenance)*/
437 INSERT INTO PRESCRIPTION_ITEM VALUES (206, 106, 1);
438
439 /*Routine Consultation (General supportive)*/
440 INSERT INTO PRESCRIPTION_ITEM VALUES (207, 113, 1);
441
442 /*Chronic Care Review (mixed long-term meds)*/
443 INSERT INTO PRESCRIPTION_ITEM VALUES (208, 105, 1);
444 INSERT INTO PRESCRIPTION_ITEM VALUES (208, 108, 1);
445
446 /*Back Pain (Pain meds)*/
447 INSERT INTO PRESCRIPTION_ITEM VALUES (209, 110, 1);
448 INSERT INTO PRESCRIPTION_ITEM VALUES (209, 112, 1);
449
450 /*Emergency Case (Strong pain + antibiotics)*/
451 INSERT INTO PRESCRIPTION_ITEM VALUES (210, 111, 1);
452 INSERT INTO PRESCRIPTION_ITEM VALUES (210, 103, 1);
453
454 /*Follow-up Visit (CNS / recovery)*/
455 INSERT INTO PRESCRIPTION_ITEM VALUES (211, 108, 1);
456
457 /*Critical Care Review (multi-system support)*/
458 INSERT INTO PRESCRIPTION_ITEM VALUES (212, 104, 1);
459 INSERT INTO PRESCRIPTION_ITEM VALUES (212, 116, 1);
```

Pharmacy Stock

```
462  /*Insert data into PHARMACY_STOCK table*/
463  INSERT INTO PHARMACY_STOCK VALUES (501, 101, 180);
464  INSERT INTO PHARMACY_STOCK VALUES (502, 102, 35);
465  INSERT INTO PHARMACY_STOCK VALUES (503, 103, 140);
466
467  INSERT INTO PHARMACY_STOCK VALUES (504, 104, 220);
468  INSERT INTO PHARMACY_STOCK VALUES (505, 105, 0);
469  INSERT INTO PHARMACY_STOCK VALUES (506, 106, 75);
470
471  INSERT INTO PHARMACY_STOCK VALUES (507, 107, 60);
472  INSERT INTO PHARMACY_STOCK VALUES (508, 108, 0);
473  INSERT INTO PHARMACY_STOCK VALUES (509, 109, 85);
474
475  INSERT INTO PHARMACY_STOCK VALUES (510, 110, 260);
476  INSERT INTO PHARMACY_STOCK VALUES (511, 111, 0);
477  INSERT INTO PHARMACY_STOCK VALUES (512, 112, 55);
478
479  INSERT INTO PHARMACY_STOCK VALUES (513, 113, 130);
480  INSERT INTO PHARMACY_STOCK VALUES (514, 114, 18);
481  INSERT INTO PHARMACY_STOCK VALUES (515, 115, 95);
482
483  INSERT INTO PHARMACY_STOCK VALUES (516, 116, 210);
484  INSERT INTO PHARMACY_STOCK VALUES (517, 117, 120);
485  INSERT INTO PHARMACY_STOCK VALUES (518, 118, 48);
```

Invoice

```
488  /*Insert data into INVOICE table*/
489  INSERT INTO INVOICE VALUES (1001, 1, 850.00, TO_DATE('2026-02-02', 'YYYY-MM-DD'));
490  INSERT INTO INVOICE VALUES (1002, 2, 1200.50, TO_DATE('2026-04-20', 'YYYY-MM-DD'));
491  INSERT INTO INVOICE VALUES (1003, 3, 450.75, TO_DATE('2026-04-02', 'YYYY-MM-DD'));
492  INSERT INTO INVOICE VALUES (1004, 4, 980.00, TO_DATE('2026-03-13', 'YYYY-MM-DD'));
493  INSERT INTO INVOICE VALUES (1005, 5, 300.25, TO_DATE('2026-01-28', 'YYYY-MM-DD'));
494  INSERT INTO INVOICE VALUES (1006, 6, 1500.00, TO_DATE('2026-05-01', 'YYYY-MM-DD'));
495  INSERT INTO INVOICE VALUES (1007, 7, 620.40, TO_DATE('2026-03-19', 'YYYY-MM-DD'));
496  INSERT INTO INVOICE VALUES (1008, 8, 275.90, TO_DATE('2026-02-10', 'YYYY-MM-DD'));
497  INSERT INTO INVOICE VALUES (1009, 9, 890.00, TO_DATE('2026-04-15', 'YYYY-MM-DD'));
498  INSERT INTO INVOICE VALUES (1010, 10, 1100.00, TO_DATE('2026-03-05', 'YYYY-MM-DD'));
499  INSERT INTO INVOICE VALUES (1011, 11, 500.00, TO_DATE('2026-05-05', 'YYYY-MM-DD'));
500  INSERT INTO INVOICE VALUES (1012, 12, 750.00, TO_DATE('2026-01-15', 'YYYY-MM-DD'));
```

Payment

```
503 /*Insert data into PAYMENT table*/
504 INSERT INTO PAYMENT VALUES (2001, 1001, 'Card', 'Paid', TO_DATE('2026-02-02','YYYY-MM-DD'));
505 INSERT INTO PAYMENT VALUES (2002, 1002, 'Cash', 'Paid', TO_DATE('2026-04-20','YYYY-MM-DD'));
506 INSERT INTO PAYMENT VALUES (2003, 1003, 'EFT', 'Paid', TO_DATE('2026-04-02','YYYY-MM-DD'));
507 INSERT INTO PAYMENT VALUES (2004, 1004, 'Medical Aid', 'Paid', TO_DATE('2026-03-13','YYYY-MM-DD'));
508
509 INSERT INTO PAYMENT VALUES (2005, 1005, 'N/A', 'Pending', NULL);
510 INSERT INTO PAYMENT VALUES (2006, 1006, 'N/A', 'Pending', NULL);
511 INSERT INTO PAYMENT VALUES (2007, 1007, 'Card', 'Paid', TO_DATE('2026-03-19','YYYY-MM-DD'));
512 INSERT INTO PAYMENT VALUES (2008, 1008, 'N/A', 'Pending', NULL);
513
514 INSERT INTO PAYMENT VALUES (2009, 1009, 'EFT', 'Paid', TO_DATE('2026-04-15','YYYY-MM-DD'));
515 INSERT INTO PAYMENT VALUES (2010, 1010, 'Card', 'Paid', TO_DATE('2026-03-05','YYYY-MM-DD'));
516 INSERT INTO PAYMENT VALUES (2011, 1011, 'N/A', 'Pending', NULL);
517 INSERT INTO PAYMENT VALUES (2012, 1012, 'Cash', 'Paid', TO_DATE('2026-01-15','YYYY-MM-DD'));
```

Emergency Contact

```
520 /*Insert data into EMERGENCY_CONTACT table*/
521 /*Patient 501 (John Doe)*/
522 INSERT INTO EMERGENCY_CONTACT VALUES (1, 501, 'Michael Liebenberg', '0825551001');
523 INSERT INTO EMERGENCY_CONTACT VALUES (2, 501, 'Linda Doe', '0831112222');
524
525 /*Patient 502 (Jane Smit)*/
526 INSERT INTO EMERGENCY_CONTACT VALUES (3, 502, 'Peter Smit', '0834442002');
527
528 /*Patient 503 (Zanele Khumalo)*/
529 INSERT INTO EMERGENCY_CONTACT VALUES (4, 503, 'Nomsa Khumalo', '0713333003');
530
531 /*Patient 504 (Kevin Louw)*/
532 INSERT INTO EMERGENCY_CONTACT VALUES (5, 504, 'David du Toit', '0811115005');
533
534 /*Patient 505 (Mary Peters)*/
535 INSERT INTO EMERGENCY_CONTACT VALUES (6, 505, 'Anna Prinsloo', '0822223333');
536 INSERT INTO EMERGENCY_CONTACT VALUES (7, 505, 'James Peters', '0846666006');
537
538 /*Patient 506 (Brian Hendricks)*/
539 INSERT INTO EMERGENCY_CONTACT VALUES (8, 506, 'Bianca Hendricks', '0797777007');
540
541 /*Patient 507 (Ayesha Patel)*/
542 INSERT INTO EMERGENCY_CONTACT VALUES (9, 507, 'Imran Patel', '0828888008');
```

Sequence Test Inserts

This section verifies that sequences are working correctly by inserting test records using NEXTVAL and CURRVAL. These tests confirm that automatically generated IDs correctly link related records across multiple tables such as Person, Patient, Staff, Appointment, and Billing(Invoice and Payment).

```
545 -- =====
546 -- 6. SEQUENCE TEST INSERTS
547 -- =====
548 INSERT INTO PERSON
549 VALUES (seq_person.NEXTVAL, 'Test', 'Patient', '0820009999', 'test@gmail.com', 'P');
550
551 INSERT INTO PATIENT
552 VALUES (seq_person.CURRVAL, 'Test Address', TO_DATE('2000-01-01', 'YYYY-MM-DD'));
553
554 INSERT INTO PERSON
555 VALUES (seq_person.NEXTVAL, 'Test', 'Staff', '0830000000', 'staff@gmail.com', 'S');
556
557 INSERT INTO STAFF
558 VALUES (seq_person.CURRVAL, 1, 1);
559
560 INSERT INTO APPOINTMENT
561 VALUES (
562     seq_appointment.NEXTVAL, 501, seq_person.CURRVAL, SYSDATE,
563     '10:00', 'Consultation', 'Scheduled');
564
565
566 INSERT INTO CONSULTATION
567 VALUES (seq_consultation.NEXTVAL, seq_appointment.CURRVAL, 'Test Diagnosis', 'Test notes');
568
569 INSERT INTO PRESCRIPTION
570 VALUES (seq_prescription.NEXTVAL, seq_consultation.CURRVAL, SYSDATE);
571
572 INSERT INTO INVOICE
573 VALUES (seq_invoice.NEXTVAL, seq_consultation.CURRVAL, 500, SYSDATE);
574
575 INSERT INTO PAYMENT
576 VALUES (seq_payment.NEXTVAL, seq_invoice.CURRVAL, 'Card', 'Paid', SYSDATE);
```

Queries

This section contains SQL queries designed to meet NexCare's information requirements. The queries demonstrate different SQL functionalities including filtering, sorting, joins, subqueries, aggregation, and data formatting to extract meaningful insights from the database.

SELECT Statements

```
580 -- =====
581 -- 7. SELECT STATEMENTS
582 -- =====
583
584 SELECT * FROM PERSON;
585 SELECT * FROM ROLE;
586 SELECT * FROM DEPARTMENT;
587 SELECT * FROM MEDICATION_CATEGORY;
588 SELECT * FROM PATIENT;
589 SELECT * FROM STAFF;
590 SELECT * FROM APPOINTMENT;
591 SELECT * FROM CONSULTATION;
592 SELECT * FROM PRESCRIPTION;
593 SELECT * FROM MEDICATION;
594 SELECT * FROM PRESCRIPTION_ITEM;
595 SELECT * FROM PHARMACY_STOCK;
596 SELECT * FROM INVOICE;
597 SELECT * FROM PAYMENT;
598 SELECT * FROM EMERGENCY_CONTACT;
```

Based on information requirements of NexCare

Displays all completed appointments and links them with patient and consultation details to show full medical records.

```
732 SELECT
733     p.Person_ID, per.FName, per.LName, c.Diagnosis, a.App_Date
734 FROM PATIENT p
735 JOIN PERSON per ON p.Person_ID = per.Person_ID
736 JOIN APPOINTMENT a ON p.Person_ID = a.Patient_ID
737 JOIN CONSULTATION c ON a.Appointment_ID = c.Appointment_ID
738 WHERE a.Status = 'Completed';
```

Limitation of rows and columns

These queries demonstrate how to restrict output using column selection and row limitation techniques. They are used to control the amount of data returned from tables for readability and performance.

Displays only selected patient details or limits results to a specific number of records for easier viewing and testing.

```
741 /*Limitations of Rows and Columns*/
742 SELECT FName, LName, Email FROM PERSON;
```

```
744 SELECT *
745 FROM (SELECT * FROM APPOINTMENT)
746 WHERE ROWNUM <= 5;
```

Sorting

This section shows how data can be ordered using the ORDER BY clause. Sorting is applied to arrange results in ascending or descending order based on attributes such as names and appointment dates.

Organises appointment and person data in a structured order to make records easier to read and analyse.

```
749  /*Sorting*/  
750  SELECT * FROM APPOINTMENT ORDER BY App_Date DESC;
```

```
752  SELECT * FROM PERSON ORDER BY LName ASC;
```

LIKE, AND, OR

These queries demonstrate conditional filtering using SQL logical and pattern-matching operators. They allow searching for partial matches, combining conditions, and refining data retrieval.

Filters records to find specific diagnoses or appointment conditions based on multiple search criteria.

```
755  /*LIKE, AND, OR*/  
756  SELECT * FROM CONSULTATION WHERE Diagnosis LIKE '%Pain%';
```

```
758  SELECT * FROM APPOINTMENT WHERE (App_Type = 'Consultation'  
759      AND Status = 'Completed') OR Status = 'Scheduled';
```

Variables and character functions

This section demonstrates the use of string functions such as UPPER, LOWER, and concatenation. These functions are used to format and standardize text data such as patient names.

Formats patient names into consistent styles and combines first and last names into a full name.

```
762  /*Variables and Character Function*/  
763  SELECT  
764      UPPER(FName) AS FirstName_Upper,  
765      LOWER(LName) AS LastName_Lower,  
766      FName || ' ' || LName AS Full_Name  
767  FROM PERSON;
```

Round

These queries demonstrate numerical formatting using ROUND and TRUNC functions. They are used to control decimal precision in financial values such as invoices.

Displays financial values in a simplified format by removing unnecessary decimal places.

```
770  /*ROUND or TRUNC*/  
771  SELECT  
772      Invoice_ID, ROUND(Total_Amount, 0) AS Rounded_Amount  
773  FROM INVOICE;
```

Date functions

This section shows calculations using date functions such as SYSDATE, MONTHS_BETWEEN, and date subtraction.

Calculates how old invoices are and determines patient age based on date of birth.

```
776 /*Date Functions*/
777 SELECT
778     Invoice_ID, Invoice_Issued, SYSDATE AS Today,
779     TRUNC(SYSDATE - Invoice_Issued) AS Days_Old
780 FROM INVOICE;
```

```
783 SELECT
784     p.FName, p.LName,
785     TRUNC(MONTHS_BETWEEN(SYSDATE, pt.DOB)/12) AS Age
786 FROM PERSON p
787 JOIN PATIENT pt ON p.Person_ID = pt.Person_ID;
```

Aggregate functions

These queries summarize data using functions like COUNT and SUM.

Provides summary information such as total number of appointments and total revenue generated.

```
790 /*Aggregate Functions*/
791 SELECT COUNT(*) AS Total_Appointments
792 FROM APPOINTMENT;
```

```
794 SELECT SUM(Total_Amount) AS Total_Revenue
795 FROM INVOICE;
```

Group by and Having

This section demonstrates grouping data into categories and filtering grouped results.

Groups data to show totals per category and filters results to highlight only departments or patients meeting specific conditions.

```
798 /*GROUP BY and HAVING*/
799 SELECT
800     Status,
801     COUNT(*) AS Total
802 FROM APPOINTMENT
803 GROUP BY Status;
```

```
806 SELECT
807     Patient_ID,
808     COUNT(*) AS Total_Visits
809 FROM APPOINTMENT
810 GROUP BY Patient_ID
811 HAVING COUNT(*) > 2;
```

```
814 SELECT
815     s.Department_ID, d.Department_Name,
816     COUNT(*) AS Total_Staff
817 FROM STAFF s
818 JOIN DEPARTMENT d ON s.Department_ID = d.Department_ID
819 GROUP BY s.Department_ID, d.Department_Name
820 HAVING COUNT(*) > 1;
```

Join

These queries combine data from multiple tables using different types of joins.

Combines multiple tables to display related information such as patients, staff, appointments, and roles in one result.

```
823  /*JOIN query*/
824  SELECT
825      p.Person_ID, per.FName, per.LName, a.Appointment_ID, a.Status
826  FROM PATIENT p
827  JOIN PERSON per ON p.Person_ID = per.Person_ID
828  JOIN APPOINTMENT a ON p.Person_ID = a.Patient_ID;
```

```
830  SELECT *
831  FROM APPOINTMENT a
832  JOIN PATIENT p ON a.Patient_ID = p.Person_ID;
```

```
834  SELECT p.FName, p.LName, r.Role_Name, d.Department_Name
835  FROM PERSON p
836  JOIN STAFF s ON p.Person_ID = s.Person_ID
837  JOIN ROLE r ON s.Role_ID = r.Role_ID
838  JOIN DEPARTMENT d ON s.Department_ID = d.Department_ID
839  ORDER BY p.LName ASC;
```

Sub-queries (Nested queries)

This section demonstrates nested queries where one query is embedded inside another. Subqueries are used for advanced filtering.

Finds records based on comparisons such as above-average invoice amounts or low-stock medication items.

```
842  /*SUBQUERY*/
843  SELECT Invoice_ID, Total_Amount FROM INVOICE i
844  WHERE Total_Amount > (
845      SELECT AVG(Total_Amount)
846      FROM INVOICE
847  );
```

```
850  SELECT Name, Medication_ID
851  FROM MEDICATION
852  WHERE Medication_ID IN (
853      SELECT Medication_ID
854      FROM PHARMACY_STOCK
855      WHERE Stock_Quantity < (
856          SELECT AVG(Stock_Quantity)
857          FROM PHARMACY_STOCK
858      )
859  );
```

Extra Functionalities

This section includes additional analytical queries such as medication stock summaries, prescription reports, unpaid invoices, and email filtering. These queries provide deeper insights into system operations and data relationships.

Shows total medication stock grouped by category to monitor inventory levels.

```
862 /*Medication and Stock Analysis*/
863 SELECT
864     c.category_name, SUM(s.Stock_Quantity) AS Total_Stock
865 FROM medication_category c
866 JOIN MEDICATION m ON c.category_id = m.category_id
867 JOIN pharmacy_stock s ON m.medication_id = s.medication_id
868 GROUP BY c.category_name;
```

Displays prescriptions together with medication details and consultation history.

```
870 /* Prescription and medication report */
871 SELECT
872     p.Prescription_ID, m.Name AS Medication_Name, pi.Quantity, p.DateIssued, p.Consultation_ID
873 FROM PRESCRIPTION p
874 JOIN PRESCRIPTION_ITEM pi ON p.Prescription_ID = pi.Prescription_ID
875 JOIN MEDICATION m ON pi.Medication_ID = m.Medication_ID
876 ORDER BY p.Consultation_ID;
```

Identifies invoices that have not been paid or do not have completed payment records.

```
879 /*Unpaid Invoices*/
880 SELECT i.Invoice_ID, i.Total_Amount, i.Invoice_Issued
881 FROM INVOICE i
882 WHERE NOT EXISTS (
883     SELECT 1
884     FROM PAYMENT p
885     WHERE p.Invoice_ID = i.Invoice_ID
886           AND p.Payment_Status = 'Paid'
887 );
```

Finds patients using Gmail addresses for communication verification or filtering.

```
889 /* Email LIKE search */
890 SELECT FName, LName, Email
891 FROM PERSON
892 WHERE Email LIKE '%@gmail.com';
```

Allows searching for patients by surname, even if the exact spelling or case is unknown.

```
895 /* Search by last name */
896 SELECT FName, LName, Email
897 FROM PERSON
898 WHERE UPPER(LName) LIKE UPPER('%smith%');
```